

Loops and Iteration

Where We Left Off

Arrays *hold many values of one type*

Operators *compute with each value*

Functions *name a group of statements*

*Missing: a way to **repeat** statements*

Doing Something 100 Times

sum = sum + psi[0] ;

sum = sum + psi[1] ;

sum = sum + psi[2] ;

...

sum = sum + psi[99] ;

*100 lines that differ in **one number***

And if the array grows to 10,000?

What a Loop Is

*A **loop** repeats a block of statements*

*It keeps going while a **condition** is true*

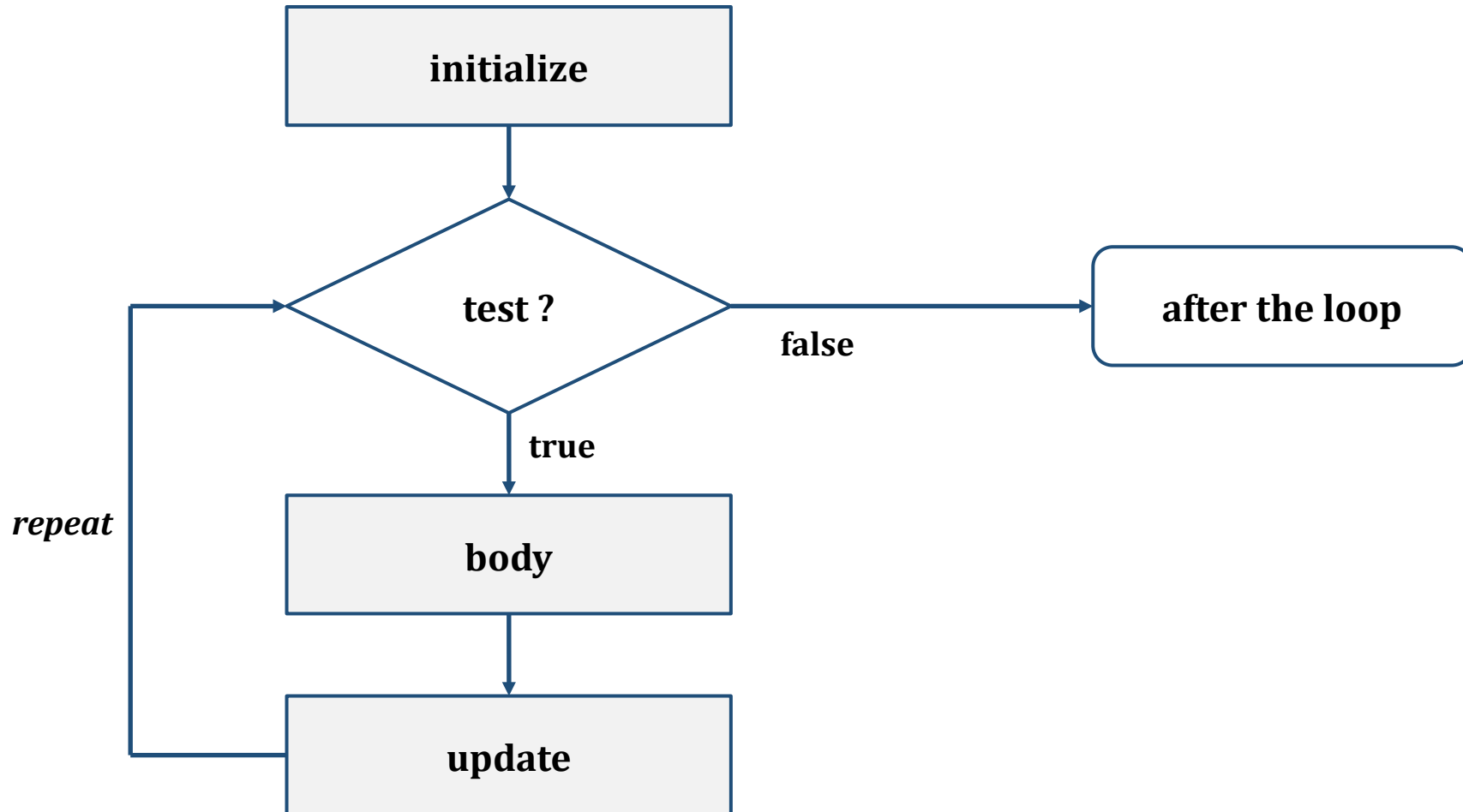
*Each pass through the block is one **iteration***

The same few lines run 100 or 10,000 times

Every Loop Has Four Parts

1. **Initialize** *set up before the first pass*
2. **Test** *should we run the body again?*
3. **Body** *the work, done once per iteration*
4. **Update** *move toward the end*

How a Loop Flows



The while Loop: C and C++

```
while ( condition ) {  
    body  
}
```

Test the condition

If true, run the body and test again

If false, continue after the closing brace

The while Loop: Rust

```
while condition {  
    body  
}
```

No parentheses needed

*Braces are **required***

*The condition must be a **bool***

Example: Count Down

C, C++:

```
int n = 10 ;  
while ( n > 0 ) {  
    printf ( "%d\n", n ) ;  
    n = n - 1 ;  
}
```

Point to the four parts

Example: Count Down in Rust

```
let mut n = 10 ;  
while n > 0 {  
    println! ( "{}", n ) ;  
    n -= 1 ;  
}
```

*n must be **mut** because the update changes it*

Example: Wait for the Sensor

```
while ( !sensor_ready ( ) ) {  
    // do nothing - just ask again  
}  
counts = read_adc ( ) ;
```

*This is **polling**: asking again and again
until the answer changes*

The Test Comes First

```
int n = 0 ;  
while ( n > 0 ) {  
    ...                never runs  
}
```

*If the condition is false at the start,
the body runs **zero** times*

do ... while: C and C++

```
do {  
    body  
} while ( condition );
```

*The body runs **first**, then the test*

*The body always runs **at least once***

Note the semicolon at the end

Example: Read Until Valid

```
do {  
    counts = read_adc ( ) ;  
} while ( counts == 0 || counts >= 4095 ) ;
```

*You must read at least once before
you can check the value*

0 and 4095 mean the sensor is open or shorted

Rust Has No `do ... while`

Use **loop** with a **break**:

```
let mut counts ;  
loop {  
    counts = read_adc ( ) ;  
    if counts > 0 && counts < 4095 { break ; }  
}
```

*The test moves to the **end** of the body*

The for Loop: C and C++

```
for ( init ; test ; update ) {  
    body  
}
```

*All the loop control on **one line***

```
for ( int i = 0 ; i < 4 ; i++ ) {  
    ...  
}
```

Tracing a for Loop

```
int a[4] = { 5, 3, 8, 2 } ;  
int sum = 0 ;
```

```
for ( int i = 0 ; i < 4 ; i++ ) {  
    sum += a[i] ;  
}
```

i	i < 4	sum
0	true	5
1	true	8
2	true	16
3	true	18
4	false	done

The body runs 4 times; the test runs 5 times.

for is while in Disguise

```
for ( int i = 0 ; i < 4 ; i++ ) {  
    sum += a[i] ;  
}
```

is the same as

```
int i = 0 ;  
while ( i < 4 ) {  
    sum += a[i] ;  
    i++ ;  
}
```

Example: Sum an Array in C

```
float psi[100];  
float sum = 0.0f;  
  
for ( int i = 0 ; i < 100 ; i++ ) {  
    sum += psi[i];  
}  
  
float average = sum / 100.0f;
```

Off by One

```
for ( int i = 0 ; i <= 100 ; i++ )  
    sum += psi[i] ;                reads psi[100]
```

*An array of 100 has indices **0 to 99***

*Use **i < N**, not **i <= N***

C and C++: reads garbage. Rust: panics.

The Loop Variable's Scope

```
for ( int i = 0 ; i < 4 ; i++ ) { ... }
```

```
printf ( "%d", i ) ;           error
```

*A variable declared in the for exists
only inside the loop*

Declare it before the loop if you need it after

The Unsigned Countdown Trap

for (size_t i = n - 1 ; i >= 0 ; i--)

*size_t is **unsigned** - it is never below 0*

After 0 comes 18,446,744,073,709,551,615

*i >= 0 is always true: the loop **never ends***

Rust Has No C-Style for

*In Rust, **for** walks through a collection of values, one at a time*

```
for i in 0..4 {  
    sum += a[i] ;  
}
```

No init, no test, no update to get wrong

Ranges

<code>0..5</code>	<code>0 1 2 3 4</code>	<i>end excluded</i>
<code>0..=5</code>	<code>0 1 2 3 4 5</code>	<i>end included</i>
<code>(0..5).rev ()</code>	<code>4 3 2 1 0</code>	<i>reversed</i>
<code>(0..10).step_by (2)</code>	<code>0 2 4 6 8</code>	<i>every other</i>

*A range is a value that produces numbers
in order*

Example: Sum an Array in Rust

```
let psi : [f32 ; 100] = [0.0 ; 100] ;  
let mut sum = 0.0 ;
```

```
for i in 0..psi.len ( ) {  
    sum += psi[i] ;  
}
```

```
let average = sum / psi.len ( ) as f32 ;
```

Iterate Over the Elements

```
for p in psi.iter ( ) {  
    sum += *p ;  
}
```

No index at all

*p **borrows** each element in turn*

**p is its contents - no bounds to get wrong*

Changing Elements in Rust

```
for p in psi.iter_mut () {  
    *p = *p * 2.0 ;  
}
```

iter_mut () borrows each element as **&mut**

psi must be declared **let mut**

Index and Value Together

```
for ( i, p ) in psi.iter ( ).enumerate ( ) {  
    println! ( "reading {} = {} psi", i, p ) ;  
}
```

*enumerate pairs each element with its index:
(0, psi[0]), (1, psi[1]), ...*

Range-based for in C++

```
std::array<float, 100> psi ;
```

```
for ( float p : psi ) {  
    sum += p ;  
}
```

C++11 and later

*p is a **copy** of each element in turn*

Changing Elements in C++

```
for ( float &p : psi ) {  
    p = p * 2.0f;  
}
```

*p is a **reference** - another name for
each element*

Without the & only the copy changes

Looping Over Structs in C++

```
std::array<sample_t, 100> history;
```

```
for ( const auto &s : history ) {  
    sum += s.psi;  
}
```

auto: *let the compiler work out the type*

const &: *read without copying 16 bytes*

Iteration vs Indexing

Indexing: *you count, then look up*
psi[i] - you can count wrong

Iteration: *the collection hands you*
each element in turn
for p in psi - nothing to count

*An **iterator** is the thing that hands them out*

One Loop, Three Languages

C: *for (int i = 0 ; i < n ; i++)*
 sum += psi[i] ;

C++: *for (float p : psi)*
 sum += p ;

Rust: *for p in psi.iter () {*
 *sum += *p ;*
 }

break: Leave the Loop Now

```
int alarm_at = -1 ;  
for ( int i = 0 ; i < n ; i++ ) {  
    if ( psi[i] > 80.0f ) {  
        alarm_at = i ;  
        break ;  
    }  
}
```

*Stop at the **first** reading over 80 psi*

continue: Skip This One

```
for ( int i = 0 ; i < n ; i++ ) {  
    if ( counts[i] == 0 || counts[i] >= 4095 )  
        continue ;  
    sum += adc_to_psi ( counts[i] ) ;  
    good++ ;  
}
```

Skip the faulty readings, keep going

Rust's loop

```
loop {  
    body  
}
```

*Repeats **forever** until a `break`*

*Clearer than `while true`
when you really mean "until I say stop"*

loop Can Give Back a Value

```
let counts = loop {  
    let c = read_adc ( ) ;  
    if c > 0 && c < 4095 {  
        break c ;  
    }  
};
```

break value *ends the loop with a result*

Infinite Loops on Purpose

*Embedded programs **never** end:*

<i>while (1) { ... }</i>	<i>C</i>
<i>for (;;) { ... }</i>	<i>C, C++</i>
<i>loop { ... }</i>	<i>Rust</i>

Arduino's `loop ( )` function is called forever by a hidden `for ( ;; )`

Infinite Loops by Accident

Forgot the update:

```
while ( n > 0 ) { printf ( "%d", n ) ; }
```

Update goes the wrong way:

```
for ( int i = 0 ; i < 10 ; i-- )
```

Test can never become false:

```
for ( double x = 0.0 ; x != 1.0 ; x += 0.1 )
```

Nested Loops

A loop inside a loop

*The inner loop runs **completely** for each pass of the outer loop*

uint8_t image[3][4]; 3 rows of 4

	<i>c = 0</i>	<i>c = 1</i>	<i>c = 2</i>	<i>c = 3</i>
<i>r = 0</i>	1	2	3	4
<i>r = 1</i>	5	6	7	8
<i>r = 2</i>	9	10	11	12

The numbers show the visiting order

Example: Nested Loops in C

```
for ( int r = 0 ; r < 3 ; r++ ) {  
    for ( int c = 0 ; c < 4 ; c++ ) {  
        image[r][c] = 0 ;  
    }  
}
```

3 x 4 = 12 iterations of the inner body

Example: Nested Loops in Rust

```
for r in 0..3 {  
    for c in 0..4 {  
        image[r][c] = 0 ;  
    }  
}
```

Or without indices:

```
for row in image.iter_mut ( ) {  
    for px in row.iter_mut ( ) { *px = 0 ; }  
}
```

Breaking Out of Two Loops

break leaves only the **innermost** loop

Rust: label the outer loop

```
'outer : for r in 0..3 {  
    for c in 0..4 {  
        if image[r][c] == 255 { break 'outer ; }  
    }  
}
```

C, C++: use a flag, or move the loops into a function and return

Looping Over an Array of Structs

```
sample_t history[100];  
float sum = 0.0f;  
  
for ( int i = 0 ; i < 100 ; i++ ) {  
    sum += history[i].psi;  
}
```

Index the array, then reach the member

Example: Min and Max in C

```
float lo = psi[0], hi = psi[0];
```

```
for ( size_t i = 1 ; i < n ; i++ ) {  
    if ( psi[i] < lo ) lo = psi[i];  
    if ( psi[i] > hi ) hi = psi[i];  
}
```

Start from the first element, not from 0

Example: Min and Max in Rust

```
let mut lo = psi[0] ;
```

```
let mut hi = psi[0] ;
```

```
for p in psi.iter ( ) {
```

```
    if *p < lo { lo = *p ; }
```

```
    if *p > hi { hi = *p ; }
```

```
}
```

A Loop Inside a Function

```
float average ( const float *psi, size_t n ) {  
    if ( n == 0 ) return 0.0f;  
    float sum = 0.0f;  
    for ( size_t i = 0 ; i < n ; i++ )  
        sum += psi[i] ;  
    return sum / (float) n ;  
}
```

Same Function in Rust

```
fn average ( psi : &[f32] ) -> f32 {  
    if psi.is_empty ( ) { return 0.0 ; }  
    let mut sum = 0.0 ;  
    for p in psi.iter ( ) {  
        sum += *p ;  
    }  
    sum / psi.len ( ) as f32  
}
```

Example: Moving Average

Smooth noisy readings: average the last W

```
for ( size_t i = W - 1 ; i < n ; i++ ) {  
    float sum = 0.0f;  
    for ( size_t k = 0 ; k < W ; k++ )  
        sum += psi[i - k] ;  
    smooth[i] = sum / W ;  
}
```

Moving Average in Rust

```
for i in ( W - 1 )..n {  
    let window = &psi[ i + 1 - W ..= i ] ;  
    let mut sum = 0.0 ;  
    for p in window.iter ( ) { sum += *p ; }  
    smooth[i] = sum / W as f32 ;  
}
```

A **slice** of W elements replaces the inner index

Example: Count the 1 Bits

```
int count = 0 ;  
while ( x != 0 ) {  
    count += x & 1 ;  
    x >>= 1 ;  
}
```

Test the low bit, shift, repeat

Rust has it built in: x.count_ones ()

Rust: Loops Without a Loop

```
let sum : f32 = psi.iter ( ).sum ( ) ;
```

```
let n_high = psi.iter ( )  
    .filter ( | p | **p > 80.0 )  
    .count ( ) ;
```

Iterator methods run the loop for you

C++: Standard Algorithms

```
#include <numeric>  
#include <algorithm>
```

```
float sum = std::accumulate (  
    psi.begin ( ), psi.end ( ), 0.0f ) ;
```

```
auto hi = std::max_element (  
    psi.begin ( ), psi.end ( ) ) ;
```

Recursion: Another Way to Repeat

*A function that calls **itself***

```
unsigned long factorial ( unsigned n ) {  
    if ( n <= 1 ) return 1 ;  
    return n * factorial ( n - 1 ) ;  
}
```

*Every problem needs a **stopping case***

Iteration or Recursion?

*Each recursive call pushes a **stack frame***

factorial (5) uses 5 frames

*A loop uses **one***

*Too deep: **stack overflow***

*Prefer a loop unless the problem is
naturally recursive - trees, for example*

Loops in the Debugger

break main.c:12 if i == 97

stop only on the iteration you care about

watch sum

stop every time sum changes

display i

print i automatically at every stop

Loop Invariants

*What is **always true** at the top of each pass?*

sum holds the total of $\text{psi}[0] \dots \text{psi}[i-1]$

True before the first pass ($\text{sum} = 0, i = 0$)

Kept true by each pass

At exit, $i = n$ - so sum is the whole total

Choosing a Loop

Every element	<i>Rust for p in ... C++ for (p : ...)</i>
Known count	<i>for</i>
Until a condition	<i>while</i>
At least once	<i>do ... while / Rust loop + break</i>
Forever	<i>while (1) / loop</i>

Loops Across the Languages

	<i>C</i>	<i>C++</i>	<i>Rust</i>
while	<i>yes</i>	<i>yes</i>	<i>yes</i>
do while	<i>yes</i>	<i>yes</i>	<i>loop + break</i>
for (; ;)	<i>yes</i>	<i>yes</i>	<i>no</i>
for each	<i>no</i>	<i>for (x : c)</i>	<i>for x in c</i>
loop	<i>no</i>	<i>no</i>	<i>yes</i>
labeled break	<i>no</i>	<i>no</i>	<i>yes</i>

What We Can Now Do

Repeat *statements with while, do, for, loop*

Iterate *over every element without an index*

Control *the loop with break and continue*

Nest *loops for tables and images*

Check the four parts: init, test, body, update

Let's practice

*Exercise: Using **looppractice***